ARCHITECTURE WRITE-UP

Governed Ticket-to-Implementation Support

A multi-agent engineering pipeline with scoped tools, one deterministic step, server-side enforcement and a human approval before every commit.

Vijayalakshmi Ramu | October 2026 | github.com/Viji-qmofi/first-microservices-Viji-R-agentic-harness-cicd

THE SYSTEM IN SIX LINES

1. An Orchestrator splits a development task among five scoped subagents and checks each result itself.
2. One step, test interpretation, is deterministic code.
3. A human approves before every commit.
4. Every tool call passes through one of three MCP servers that enforce a per-role allow-list on the server side.
5. CI blocks changes that weaken those controls.
6. Section 9 lists what this does not yet prove.

Every statement below is taken from a file in this repo, cited in brackets. Where the repo does not say something, this document says so.

01 Purpose and workflow

This system helps one developer turn a ticket into a reviewed, tested code change on a real Spring Boot microservices project (four services: eureka registry, API gateway, product service, order service). The Module 1 build check covers all four services. The planner and implementer are written for one of them, the order service, so orchestrated code changes are scoped there. The developer stays in control: an Orchestrator splits the work among specialist agents, checks each result itself instead of trusting the agent's own summary, and stops for a human decision before anything is committed [CLAUDE.md, Orchestrator Instructions].

The path of a normal task is:

1. A `planner` agent retrieves relevant prior lessons and produces a numbered plan and file list, with no code.
2. The Orchestrator reviews the plan and either sends it back, asks the human to approve an externally visible behavior change, or hands it on.
3. An `implementer` agent writes the code exactly per the approved plan.
4. The Orchestrator checks that the diff matches the claimed scope.
5. A deterministic script reports the test result.

6. One or two reviewer agents review the diff independently.
7. The human approves, and only then is the change committed.

A fifth subagent, `decision-auditor`, is not part of this path. It is invoked separately to check recorded project decisions against real git state.

02 System at a glance

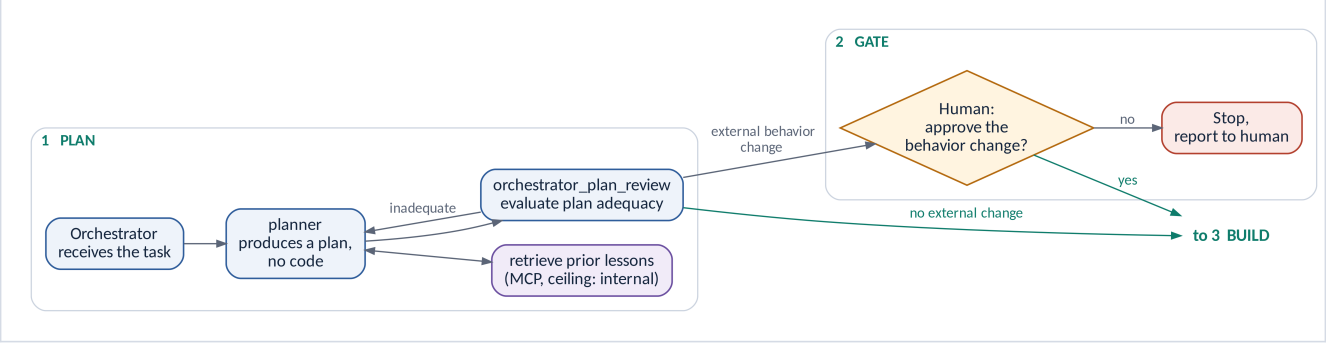
The pieces, from the outside in:

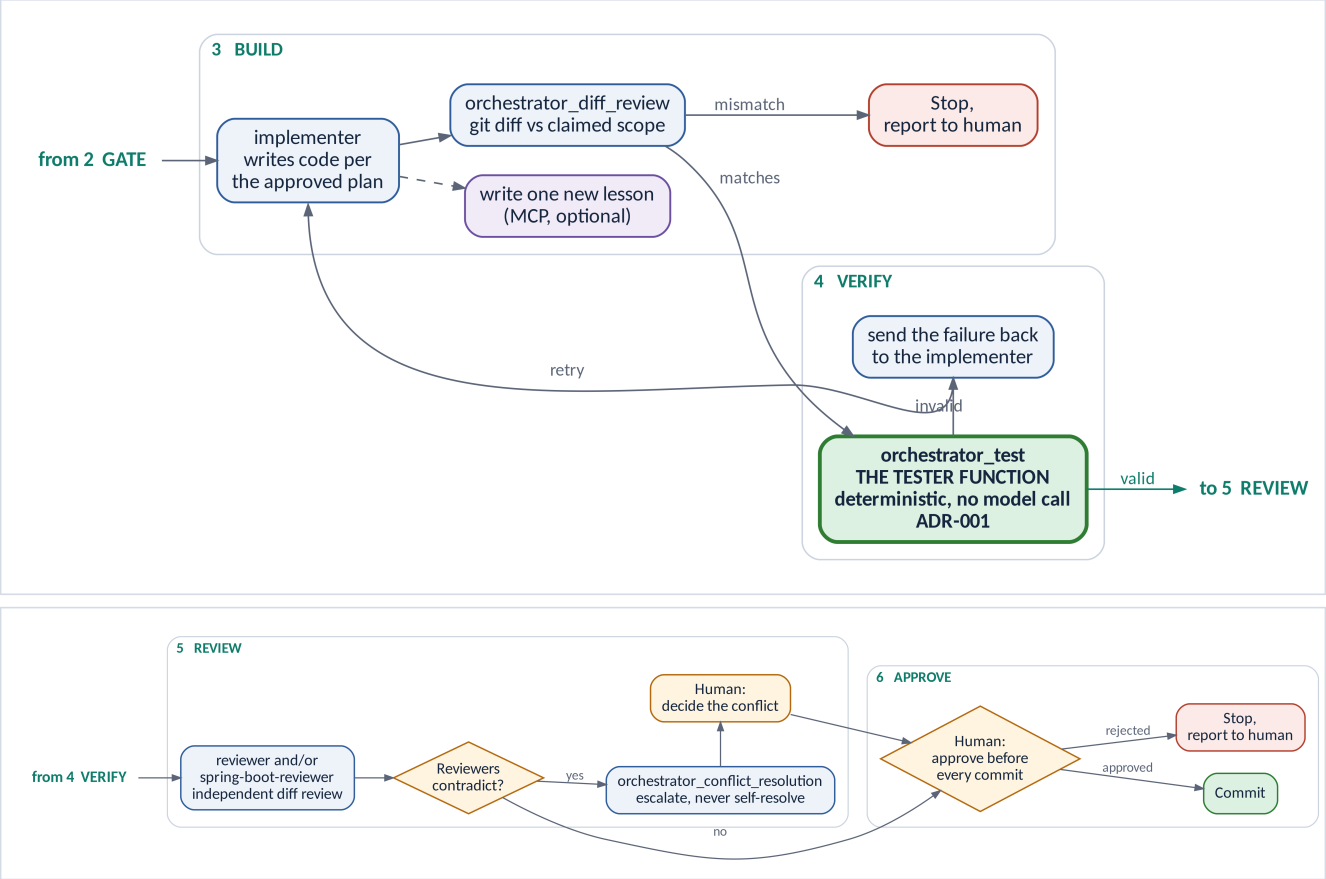
- **A Docker container** is the working environment. It mounts the project workspace, keeps read-only knowledge files read-only with `--cap-drop=DAC_OVERRIDE` and a `:ro` mount, and persists the Claude Code login in a separate volume [CLAUDE.md, Docker Commands].
- **The Orchestrator** is the top-level Claude Code session. It invokes and evaluates subagents, reviews diffs and runs tests. It never writes code itself [CLAUDE.md].
- **Five subagents** with defined jobs and tool grants: `planner`, `implementer`, `reviewer`, `spring-boot-reviewer`, `decision-auditor` (section 3).
- **One deterministic step**, `orchestrator_test`, which uses no model (section 4).
- **Three MCP servers** that every governed tool call passes through: `coursetools` (files and search), `storage` (project-memory operations) and `retrieval` (vector search over reference lessons). Each enforces a per-role allow-list [CLAUDE.md, MCP Servers].
- **Three memory layers** under `.memory/` (project, knowledge, reference) [CLAUDE.md, Memory Configuration]. Section 6 covers these.
- **An evaluation harness** (deterministic checks plus a rubric-scored judge) and **a CI pipeline** that enforces policy and integrity on every pull request. Sections 6 to 8 cover these.

The full flow, including every checkpoint and the MCP boundary layer, is in `docs/orchestration-diagram.md`. `docs/orchestration-diagram.md` (Mermaid) is the source of record. Figure 1 in this document is a redrawn copy of it, split into three rows so the labels stay readable on a page.

Figure 1. Orchestration flow

One task from receipt to commit, read in three rows: plan and gate, build and verify, review and approve. `docs/orchestration-diagram.md` (Mermaid) is the source of record; this is a redrawn copy so the labels stay readable.





AGENT STEP
A role does judgment work. The role is named in the box.

DETERMINISTIC
Code, no model call: the tester function (ADR-001).

HUMAN DECISION
Two separate decisions: the conflict, and the commit.

STOP
Every stop ends in a report to the human.

MCP TOOL CALL
Checked against a per-role allow-list.

Separate from this path, **decision-auditor** runs on demand to check project records against git. Roles are self-declared, so the allow-lists are policy, not proof of identity (section 7).

03 Agent roles

Each role below is described by its job, what it may use, and what it is deliberately denied. The authoritative source is `docs/governance-policy.md`, which is machine-checked against the allow-lists and agent files [docs/routing-and-tool-grant-map.md].

> **`planner`**
> Decides how to make a change and produces a plan. May read files and search the codebase, and may retrieve prior lessons up to the `internal` classification. May not write anything. The Orchestrator checks that its plan cites what it retrieved, or says that nothing relevant existed.

`implementer`
Writes code exactly per an approved plan. May read, write and search files, and may add one new lesson through `write_entry`. It cannot read, list or edit existing lessons, and has no retrieval access, because the planner already supplies retrieved context. It also has no test-running access, by design.

`reviewer`
Independent, advisory diff review for correctness, test coverage and scope. May read and search files only. Storage and retrieval grants it once had were removed under least privilege, because its agent file never used them (ADR-007).

`spring-boot-reviewer`
Advisory review for Spring Boot conventions: exception handling, REST conventions, Feign usage and missing tests. It uses Claude Code's native Read, Grep, Glob and Bash tools and has no MCP grants. It predates the MCP storage and retrieval layer, so it is outside that layer's enforcement.

`decision-auditor`
Checks and corrects `.memory/project/` records against real git state. It is the only role granted `update_entry`, and it has a file read and write exception limited to `.memory/project/`. It is denied `retrieve` on purpose: every correction must trace to something it verified directly, never to a remembered lesson.

`orchestrator`
Coordinates, evaluates and runs shell commands such as `./mvnw test`. May read and list storage entries, read the audit log, and retrieve up to the `confidential` classification. Its `write_entry`, `update_entry` and `delete_entry` are explicitly denied. That denial is not theoretical: a calibration run (HO-06) recorded the Orchestrator successfully calling `update_entry` before server-side enforcement existed [docs/routing-and-tool-grant-map.md]. One limit applies: the Orchestrator's own native file-editing tools are not subject to the MCP role checks, so "never writes code" is an instruction in `CLAUDE.md`, not a technical block [CLAUDE.md].

Roles we deliberately do not have

There is no tester role. Test execution is a step, not a role. Every task ends with `orchestrator_test`, deterministic code that runs `./mvnw test` and returns a structured pass or fail from the Surefire summary (ADR-001: about 264x lower latency, zero token cost, byte-identical output). Writing tests belongs to `implementer`, and judging their adequacy belongs to `reviewer`. A separate tester agent would duplicate the reviewer's judgment and add a role with its own grants and attack surface, so ADR-006 rejects it. The trade-off is that there is no agent doing exploratory test design. At this scale the deterministic step plus review covers it.

04 Workflow steps and the right tool for each

Every step is classified as agentic, deterministic or human, with a reason. Classification is reviewed on a schedule, and a step becomes a conversion candidate only when it is stable, repeatable, specifiable and frequent [docs/step-classification.md].

AGENTIC judgment and synthesis needed

- `planner`: output is not specifiable in advance; the reasoning differs for every task.
- `implementer`: writing code per a plan requires judgment about implementation in context.
- `reviewer` and `spring-boot-reviewer`: reviewing arbitrary changes requires reading code in context.
- `decision-auditor`: each decision record needs different verification logic against different git evidence.

ORCHESTRATOR JUDGMENT reading free text

- `orchestrator_plan_review`: judging whether an open-ended plan is concrete, correctly scoped and in-role.
- `orchestrator_diff_review`: checking that the diff matches the plan's intent. It is a weak conversion candidate, and a narrow part of it (line-ending noise) has a readiness package and a proposed ADR-002, not an implementation. A real counterexample from this project, the `docker-entrypoint.sh` line-ending fix, shows why full automation would be wrong there.
- Reviewer conflict resolution: it exists to send a real disagreement to a human, so a fixed rule would defeat its purpose.

DETERMINISTIC a fixed rule is enough

- `orchestrator_test`: converted from an agentic step to code (ADR-001). It reads Maven's summary line and keys only on its counts, never on the words ERROR or WARN elsewhere in the log, because the test suite deliberately produces both. If no summary line exists, for example after a compile failure, the result is explicitly `valid: false`, never a silent pass.
- `_authorize()` role checks on all three MCP servers and `validate_classification()` on storage writes.
- Pipeline gates in CI (section 8).

HUMAN

- The three checkpoints in section 5.

05 Human checkpoints and failure handling

Three human checkpoints, with explicit triggers, not a blanket "ask every time":

1. **Behavior change.** If a plan proposes a change visible outside the codebase, such as a change to API responses, the Orchestrator asks the human for approval before implementation, even if the plan is sound.
2. **Reviewer conflict.** If two reviewers return contradictory verdicts on the same section, the Orchestrator may not pick one or average them. It records an `orchestrator_conflict_resolution` step, sets the transcript's `escalated_to_human` flag, presents both full reasonings and waits for an explicit human decision.
3. **Before every commit.** The Orchestrator presents the plan, diff and test result and waits for approval, regardless of whether the run went smoothly.

CLAUDE.md defines the second and third as separate rules, and the orchestration diagram shows them as two separate human nodes: the conflict decision, and then the approval before every commit. An advisory review of this repo flagged the earlier single-node version as ambiguous.

The Orchestrator stops, and does not continue on its own, when:

- the diff does not match the claimed scope;
- the test script returns `valid: false`, in which case it reports the `problems` list to the human;
- a subagent uses a tool outside its grant or produces output outside its job;
- a plan has been sent back for revision three times, in which case it asks the human to clarify the task;
- the task's own cost exceeds $2.00 or its wall-clock time exceeds 900 seconds, in which case it reports the overage and asks whether to continue, scope down or abandon;
- a required MCP server is unreachable, in which case it never offers to proceed without it and the only options are to start the server and restart the session, or to abandon the task;
- a subagent has not returned after 10 minutes, which it treats as a stall.

> **Not yet specified.** CLAUDE.md covers a test script that returns `valid: false`. It does not say what the Orchestrator should do if the script itself crashes or is missing. A crash would most likely surface as a non-zero exit, but that is not documented, so it is listed as an open risk in section 9.

06 Data flow, memory, storage and retrieval

Memory layers under `.memory/` [CLAUDE.md, Memory Configuration; module2_doc/memory-architecture.md; ADR-004]:

- `project/` holds decision records and `MEMORY_INDEX.md`, which the Orchestrator reads at the start of every session. Entries carry review dates, and an entry that no longer describes a real decision is moved to `archived/` with a documented reason.
- `knowledge/` holds stable conventions such as the coding standards. It is mounted read-only into the container, with `--cap-drop=DAC_OVERRIDE`, because a `chmod` alone does not stop root.
- `reference/` holds the lesson documents that retrieval searches. Agents reach it through the retrieval server, not by listing the directory.

Storage server [mcp-servers/storage]:

- It is backed by SQLite and exposes six named operations: `write_entry`, `read_entry`, `list_entries`, `update_entry`, `delete_entry` and `audit_read`. It exposes no raw SQL.
- Every operation is scoped to a `project_id`. Classifications are `public`, `internal`, `confidential` and `secret`, and writes are accepted only at `public` or `internal`, so a secret cannot be stored through this path.
- Every write-class operation appends one line to an audit log: timestamp, operation, calling role, entry id, project and classification.

Retrieval server [mcp-servers/retrieval; module3_doc/retrieval-quality-report.md]:

- It has one operation, `retrieve`. Each reference document is split into chunks and embedded with `all-MiniLM-L6-v2`, with a `sqlite-vec` vector index and an FTS5 keyword fallback.

- Every result carries `source_document`, `chunk_index`, `excerpt` and `classification`, so each answer can be cited. A document with no valid classification is indexed as `secret`, so a mistake hides it and does not leak it.
- The effective classification ceiling is the lower of what the caller requests and what its role is allowed. `planner` is capped at `internal` and `orchestrator` at `confidential`. No other role may retrieve.
- Quality was checked against a six-query ground-truth set: 5 of 6 passed (83%), above the 80% floor. The one failure (Q4) is a documented known gap: the right document came back only through the keyword fallback, which carries no similarity score.

Typical data flow. The `planner` calls `retrieve` for prior lessons and cites what it found. The `implementer` may add one lesson with `write_entry`. The Orchestrator may read entries and the audit log, but may not write. `decision-auditor` alone may correct existing project records.

07 MCP boundaries and governance enforcement

Enforcement sits in three places, so no single layer is trusted alone (ADR-005):

1. **Container permissions.** Read-only mount and dropped capability for `knowledge/`. `scripts/run-agent.ps1` launches a role with only the filesystem mode its policy allows: read-write for `implementer` and `orchestrator`, read-only for the rest. In day-to-day use the subagents run inside the Orchestrator's one container, so this script is verification evidence, and the layer below is what constrains them [scripts/run-agent.ps1 header].
2. **Server-side allow-lists.** Each MCP server checks the calling role against its own allow-list and denies by default: storage `_authorize()`, retrieval's per-role ceiling, and `coursetools`, which also blocks `.memory/` except for `decision-auditor`'s scoped exception. `coursetools` refuses to start if its allow-list is missing. The grants are in `docs/routing-and-tool-grant-map.md` and `docs/governance-policy.md`.
3. **Tests and gates.** `eval/test_policy.py` and `eval/test_governed_files.py` compare the written policy with the real allow-lists, agent files and skills. The evaluation harness checks recorded transcripts for forbidden operations. Red-team prompts probe the boundaries: all four planned prompts held, and one organic finding is recorded separately [eval/red-team-results.md]. A pre-commit hook (`scripts/hooks/pre-commit`) blocks a commit when a staged file under `.memory/` contains a credential-shaped string such as `password=` or `sk-`. It is local and must be activated once per clone with `git config core.hooksPath scripts/hooks` [setup.md].

Evidence that it bites. The Orchestrator's `update_entry` call succeeded before enforcement existed, recorded in the audit log at `.eval-artifacts/holdout-run-1/HO-06.log`, line 2. The identical call is now denied, verified with a real MCP call [eval/enforcement-verification.md].

Known limits, stated plainly:

- Roles are self-declared. The server enforces the allow-list against the role a caller states, and nothing verifies who is calling. A direct test showed the Orchestrator writing with `calling_role="implementer"` successfully. The `.memory/project/` exception for `decision-auditor` rests on that same claimed role, although the path limit itself is a real server-side check.
- `spring-boot-reviewer` uses native Bash and sits outside the MCP layer. The Orchestrator's native file-editing tools are also outside it.
- `coursetools` keeps no audit log of its own.

08 CI/CD integration

The pipeline is `.github/workflows/ci.yml` [docs/ci-step-design.md]. A classifier job reads the changed paths and decides which conditional gates do real work. Every job still starts on every pull request, so a required check can never be left pending.

Policy Test Suite REQUIRED	Structural checks on the repo and workflow.
Governed File Check REQUIRED CONDITIONAL	Declared grants against configured grants, for changes to agents, skills, allow-lists, `eval/`, the policy and the routing map.
Evaluation Harness REQUIRED CONDITIONAL	Replays a committed known-good transcript (HO-02) through the deterministic and rubric checks. It shows the harness still reproduces a good result. It does not evaluate the behavior of a changed prompt.
Pipeline Integrity Check REQUIRED	It inspects `ci.yml` itself and fails if a gating job gains `continue-on-error`, if `audit-trail` loses `if: always()`, or if the classifier disappears. It has no `needs:` so that weakening one job cannot also skip this check.
Advisory Code Review ADVISORY	An advisory model review of changed files on pull requests. It reports clearly when it could not run.
Audit Trail ALWAYS RUNS	One JSON record per run, kept 90 days.

Main requires a pull request and the four required checks. Every job has a hard timeout. Image builds retry three times and then fall back to a mirror of the base image.

Evidence of gating. Weakening a gate on a throwaway pull request turned the required Pipeline Integrity Check red (PR #9), and the base-image fallback was exercised with a simulated failure (PR #10). On a real pull request the Evaluation Harness failed closed when the model API's credit balance ran out, and stayed red until credit was added. That incident also exposed that the advisory review had reported success while failing, which was then fixed. Two deliberate tests on throwaway pull requests, closed without merging, exercised the controls: with the Policy Test Suite limit set to 1 minute, GitHub cancelled the job mid-build and the dependent checks were skipped (PR #23, run 38971124589); with the cost budget lowered to $0.50, HO-02's real $0.95 cost failed the required Evaluation Harness while every other check passed (PR #24, run 38972843072).

09 Decisions and open risks

Decision records (`docs/adr/`):

ADR-001	the test result step converted to deterministic code (about 264x faster, no token cost).
ADR-002	a second conversion (line-ending noise), readiness package only, not implemented.
ADR-003	per-dimension floor for the quality rubric.
ADR-004	three-layer memory.
ADR-005	MCP boundaries enforced server-side.
ADR-006	the smallest role set that maps to real responsibilities, including no tester role.
ADR-007	governance policy built from near-miss evidence.

Open risks, consolidated:

- Roles are self-declared, not verified identities (section 7).
- `coursetools` has no audit log, and two agents use native tools outside MCP enforcement.
- CI sets no explicit `permissions:` block, so jobs use the repository's default token permissions.
- The integrity checker can be edited by the same pull request it checks, so that pull request needs human review.
- The Evaluation Harness replays a frozen transcript, so it does not test the effect of a changed prompt.
- The $2.00 and 900-second budgets and the three-revision limit are written instructions to the Orchestrator for live runs. The after-the-fact budget check has fired, in CI on a deliberately lowered ceiling (PR #24), but the live guards have not been shown stopping a real run.
- The image-build fallback has been exercised only with a simulated failure.
- If the test-result script itself crashes or is missing, the documented behavior is undefined (only `valid: false` is covered).
- The Module 1 prompt fix is inconsistent (3 runs), and review latency has one timed observation. Section 6 of `module4_doc/module1-comparison.md` explains both.
- The secret-scanning hook covers `.memory/` only, and protects only clones that have activated it. Nothing in CI checks that it is active.
- The `planner` and `implementer` definitions are written for `ecom-order-service`. A task in another service needs new scoped definitions. In a rehearsal the Orchestrator refused to run them on the product service and asked first.
- Only a Windows launcher exists for the container.

What this document does not claim

- No production deployment, no live team, and no customer data. The workload is a representative four-service application, and the orchestrated roles are scoped to its order service.
- Governance did not improve accuracy on the Module 1 build-check task. Its value is in blocked actions, audit and human approval, not in that score.
- The role boundaries are policy enforced against declared roles, not proof of caller identity.
- Network egress is restricted by default with a documented exception for sessions that call the model. It is not claimed airtight.

Source evidence

`CLAUDE.md`, `docs/orchestration-diagram.md`, `docs/routing-and-tool-grant-map.md`, `docs/governance-policy.md`, `docs/step-classification.md`, `docs/ci-step-design.md`, `docs/adr/`, `mcp-servers/`, `eval/enforcement-verification.md`, `eval/red-team-results.md`, `module3_doc/retrieval-quality-report.md`, `module3_doc/calibration-log.md`, `.eval-artifacts/holdout-run-1/`.